cleversafe®

an IBM Company

Aspen

Tom Cocagne

IBM/Cleversafe Proprietary

Introduction

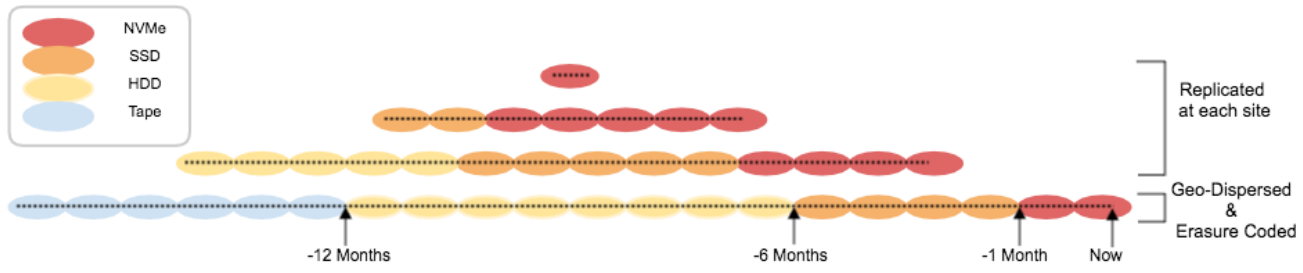
In 2005 I started working on a project intended to distribute erasure-coded file and backup data throughout K-12 computer labs. With poor funding and a lack of trained IT staff being the norm rather than the exception, reliable data management was a major pain point in those environments. The concept behind the project was that the desktop machines comprising most computer labs already had more than enough raw storage capacity to satisfy their storage needs; it just needed to be used effectively. Of course, developing a distributed, erasure-coded storage system is a non-trivial task to begin with but this system additionally faced the challenges of needing to operate in a dynamic environment consisting of an amalgam of heterogeneous and often cobbled-together hardware with unusually high rates of failure. A practical solution escaped me at the time but it was an interesting technical challenge and has remained an occasional side project ever since. In the last few years the pieces have started coming together and, surprisingly, the resulting system looks as though it may be well suited to addressing some of the challenges currently present in the cloud environment.

Aspen, so named for reasons only a dendrologist is likely to appreciate, is a distributed data architecture specifically designed to support the creation of highly customizable, distributed data structures. While this may not look like much at first glance, a case in point of the utility of such structures is the Elasticsearch search engine. Elasticsearch ingests unstructured data, parses it into keywords, and outputs the parsed content into a set of distributed indices that enable fast queries across the stored content. Of particular note is that the wild success of Elasticsearch is not due to the development of a new search algorithm specifically tailored to unstructured, big data systems. Rather, it uses a fast, distributed version of the indexing data structure underlying the existing Lucene search engine; an engine that was originally designed for use on only a single machine.

Aspen generalizes upon the notion of a distributed index to enable the creation of arbitrary distributed data structures and additionally provides a number of options for tuning those structures for peak performance under specific work flows and operational environments. Though seemingly counter-intuitive, scalability and performance in Aspen is achieved through the use of design concepts explicitly avoided by most scalable systems in use today. In particular, Aspen is based on explicit data placement and object pointers with embedded location information instead of the more common sharding, namespacing, or distributed hash table mechanisms. Whereas the latter systems scale by hiding and abstracting location information away from the application, Aspen scales by exposing it as a design parameter.

For example, consider the admittedly contrived case of managing large-scale IoT sensor data where high-throughput, low-latency access is needed for new data but access to older data can tolerate lower performance for the benefit of storing it on less expensive media. An Aspen-based system could directly support this use case with a distributed tree structure that uses data placement and migration policies tuned specifically for this workflow. The sensor data may be erasure-coded and dispersed between multiple geographic sites for advantageous storage efficiency and availability characteristics while indices over that data may be replicated at each site. This would allow for fast local lookups without incurring the latency penalties associated with over-the-wan access to each tier of the dispersed tree or the excessively large in-memory cache of tree nodes that would otherwise be required. To further reduce lookup latency, the upper tiers of the tree structure may be backed by high-performance media, such as NVMe or SSD, while the sensor data itself is spread over a variety of backing media based on the age of the data.

Copyright © 2016 Cleversafe, Inc.



Carrying the example a bit further, efficient searches based on geographic location could be achieved by placing pointers into a distributed R-Tree, which is ideally suited for such queries. Distributed queues could be used for delivering data processing tasks to workers and distributed graphs could be generated by those tasks to link together sensor data with similar characteristics. Furthermore, in situ parallel processing algorithms like map-reduce could be used to speed up the execution of those tasks if replication were used instead of erasure-coding.

Modifications of objects in Aspen data structures use immediately-consistent, ACID-compliant transactions that are capable of updating multiple objects at once. Additionally, transactions may specify multiple follow-on actions that must occur after a successful transaction is committed. Overall, the transaction model is similar to that planned for a future version of IBM's Cloud Object Storage system but Aspen places a much stronger emphasis on durability and ensures that crashes at any point in time cannot corrupt objects, lose data, or renege on successes returned to the client. The strong transaction model is of critical importance to Aspen's overall architecture as this is largely a self-hosting system. All of the data integrity and error recovery mechanisms are based upon the notion of using distributed data structures to track failures and drive error recovery processes.

Aspen represents a distinct departure from the current state-of-the-art in distributed systems technology in so far as there do not appear to be any other existing systems with intents or designs that are even passingly similar to those described here. As wild-duck ideas go, this one is certainly out there. However, the potential advantage is that because this system uses a completely different paradigm for designing distributed data systems, it may open up possibilities for a whole new class of distributed data applications; ones that are not practical today given the design constraints imposed by existing systems. A few potential applications which may benefit from taking a different tack compared to contemporary solutions are distributed indexing, hadoop-like parallel processing, and massively scalable SQL/NoSQL database engines.

The current status of this project is that the concept is fairly mature but the prototype is still a work-in-progress and on its fourth implementation. The first two prototypes were useful in evolving the overall concept and design whereas the third helped in figuring out both what to do, and what not to do, from a software design and implementation perspective. The fourth prototype is coming along quite well but given the scope of the overall system, it's not something that can realistically be completed as an individual, spare-time project. My hope is that someone in the extended IBM team will find this project interesting enough to pick up and pursue in earnest. It would not be overly difficult or time consuming to build a complete system for at least sunny-day scenario evaluation purposes.

Technical Overview

At the basic level, Aspen is a distributed object storage system designed to support the construction of arbitrary, distributed data structures. In contrast with most other distributed storage systems, this architecture

Copyright © 2016 Cleversafe, Inc.

requires explicit placement of data on logical data storage endpoints. Stores are grouped into small pools from which objects are allocated and the stores within a pool may be freely migrated between physical machines and backing media. During the object allocation process, a pointer is created that specifies exactly where within each store the object data is located. This may then be handed back to the stores at at later time to retrieve the object content. These pointers may be serialized and embedded in the content of other objects to form arbitrarily complex distributed data structures. Reference counts are replicated along with each object slice/replica and the object is deleted only when a quorum of the reference counts reaches zero.

Integrity of such structures is ensured by an immediately-consistent transaction model that can be summarized in ACID terms as:

Atomic

Multiple objects and/or reference counts may be simultaneously updated in all-or-nothing transactions that have no intermediary representations.

Consistent

Paxos consensus is used to ensure that a write-threshold number of replicas and/or erasure-coded slices are updated together. Version checks at each slice/replica are further used to ensure that the pre-commit state of the objects exactly match those required by the transaction.

Isolated

The default level of isolation in Aspen is on a per-object basis due to the all-or-nothing atomicity guarantee. With respect to multiple objects, where the timing of reads can potentially return partial results of a multi-object transaction, additional isolation mechanisms must be employed to prevent inconsistencies. Two such mechanisms that are compatible with Aspen's data and transaction models are Multi-Version Concurrency Control and, the much lighter-weight, Read-Atomic Isolation.

Durable

Transactions are fully durable. Crashes at any point in time will not result in corruption or inconsistencies.

The following sections describe Aspen's data and transaction models as well as the internal support structures and error recovery mechanisms needed to ensure long-term integrity. Several areas such as specific data migration policies, network message passing, and data migration schemes have yet to be fully thought-through and defined. Consequently several such sections which would be required for a fully operational system are omitted.

System Components

From an architectural perspective, there are only four main components: Objects, Object Pointers, Data Stores, and Storage Pools. The logical relationship between these components is as follows: Object Pointers are contained within Objects, Objects are contained within Data Stores, and Data Stores are contained within Storage Pools.

Object

As with most distributed object storage systems, the primary role of Objects in this system is to hold uniquely identifiable chunks of data for retrieval and update purposes.

Copyright © 2016 Cleversafe, Inc.

Immutable Attributes

UUID

All objects in Aspen are assigned a random UUID to uniquely identify the object within the system.

ObjectSize

Aspen object allocation is similar to `malloc()` in that the size of the object is fixed at the time of allocation. Objects may use less than the total allocated space but they cannot grow beyond it.

IDA

The Information Dispersal Algorithm (IDA) describes whether the object is replicated, erasure coded, or both and the associated parameters such as replication factor and read/write threshold requirements.

Mutable Attributes

ObjectContent

The data within an object may be modified through the use of immediately-consistent, atomic transactions that either append to the current data stored within the object or overwrite it entirely.

ObjectRevision

Object revisions are a tuple of `(overwriteCount, currentLength)`. The primary uses for the object revision are to define the number of bytes currently stored in the object, which may be less than the `ObjectSize`, and to ensure that transactional updates are modifying the expected version of the object.

ReferenceCount

Reference counts on Aspen objects are employed in the traditional manner where objects are deallocated when their reference count reaches zero. Updates to the reference counts are done in an atomic manner and are independent of the object content. Simultaneous updates to both the reference count and object content by different transactions will not conflict.

LastCommitUUID

Identifies the UUID of the last transaction to successfully update the object content. This is needed by the transaction model to resolve certain error conditions.

Supported Operations

Reference Count Update

Changes the reference count to the new value specified in the transaction. These will typically involve a single increment/decrement from the current value but larger changes are possible with multi-object transactions that add or remove multiple references to the object within the scope of a single transaction.

Append Data

Data appended to the object will be placed at the offset of the `currentLength` parameter of the revision at the time of the update. The append operation will also update the revision's `currentLength` to reflect the appended content but the `overwriteCount` remains unchanged.

Overwrite Data

Overwrites completely replace the existing content of the object with the new data specified in the transaction and change the revision to `(previousOverwriteCount+1, newCurrentLength)`

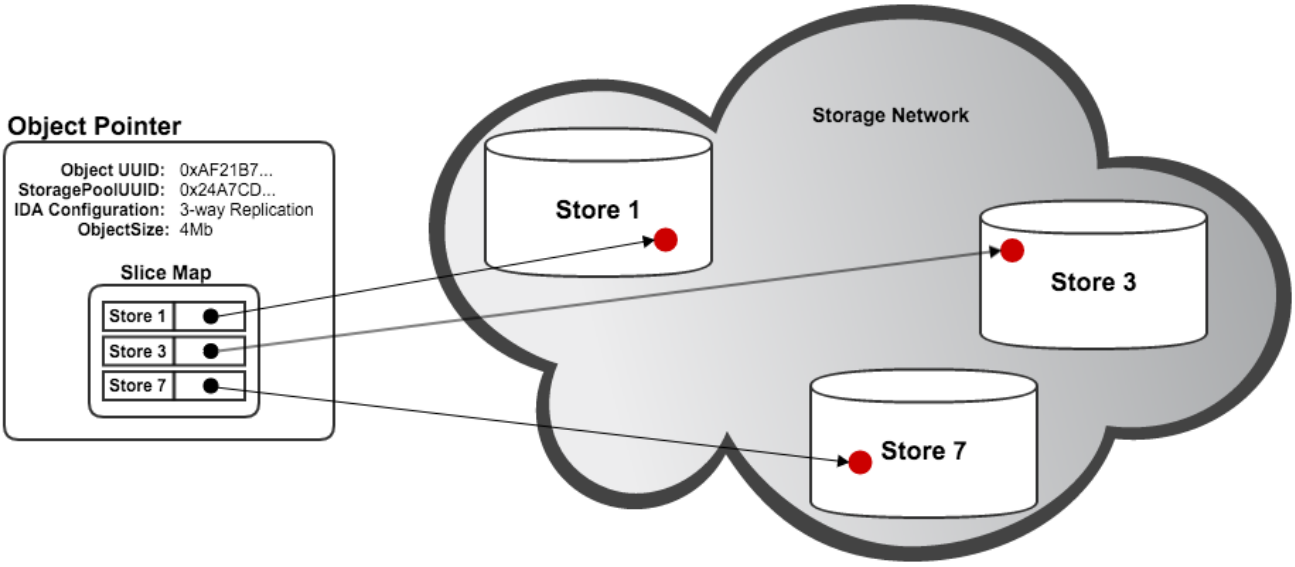
Copyright © 2016 Cleversafe, Inc.

Version Bump

Version bumps increment the `overwriteCount` of the revision but leave the object otherwise unchanged. As the transaction model section explains, this can be used to ensure the failure of any potentially outstanding transactions that are attempting to modify an object.

Object Pointer

In namespace-based object storage systems, such as Cleversafe's dsNet® technology, the physical locations of object slices/replicas can be calculated directly from the name of the object. This approach allows for extremely fast object lookups as no searching is required to discover object locations but it comes at the cost of limited flexibility in terms of object configuration, placement, and migration. To better support these characteristics, Aspen instead uses a pointer-based model where explicit pointers are used to locate object slices/replicas.



The pointers do not have a fixed size due to the variable number of replicas/slices that may be used for an object. Also, each store provides a unique token during object allocation that identifies the location of the replica/slice within the store. The token itself is treated as an opaque, binary string and is also allowed to vary in size. Generally though the serialized size of Object Pointers is relatively small and will usually fall in the range of 50 - 100 bytes.

Object Pointer Contents

ObjectUUID

Identifies the object being pointed to and is used during read/write operations to ensure that the correct object is being accessed.

StoragePoolUUID

Storage Pool the object is contained in.

IDAConfiguration

Replication/erasure-coding algorithm, width, read-threshold, and write-threshold

Copyright © 2016 Cleversafe, Inc.

ObjectSize

Maximum size of the object in bytes

SliceMap

Associative array of (Data Store ID -> `sliceToken`). The purpose of the slice map is to specify which stores host slices/replicas of this object as well as the store-specific `sliceToken` that must be used to retrieve the slice/replica from each store.

Data Store

Data Stores are logical entities that store object slices and replicas. They do not have a fixed network endpoint or required type of storage media and they are allowed to freely migrate between physical machines and backing devices to best suit the needs of the application and operating environment. The architecture does not place any limitations on the allowable size for data stores but relatively small stores will generally be preferable since smaller stores are faster to migrate and they are more likely to fit within the size constraints commonly imposed by faster physical media.

When allocating space for an object slice or replica, the store returns a token that identifies the location of the object within the store. This value is embedded within the Object Pointer and is used every time access to the slice/replica on that store is needed. The token itself is treated as an opaque value by the rest of the system and there are no hard restrictions on its allowable size. Consequently, the data store may return whatever value that best suits the underlying implementation. A store backed by a flat-file, for example, could simply return the offset of the object within the file.

Though seemingly simplistic, the ability to place offsets in object pointers makes flat-files surprisingly effective data stores. Assuming the file is stored contiguously on disk, locating the object is virtually instantaneous and accessing the data requires only a single head swing when the store is backed by rotating media. Also, there is virtually no memory or cpu overhead which makes them suitable for resource-constrained hosting machines.

Another option for data store implementation is a key-value database such as LevelDB where an opaque key, rather than an offset, is stored in the object pointer. Such a store would avoid the wasted space that is inherent to contiguous, pre-allocated files when utilization is low but comes at the cost of additional lookup latency and higher resource requirements. Fortunately though, data stores are not necessarily locked in to any particular implementation. Just as stores may be migrated between machines and physical media, so too may their data be migrated between differing implementations so long as the tokens stored within the object pointers remain valid. A store could, for example, start off with a flat-file implementation and later be migrated to a LevelDB database after dropping below a certain utilization threshold. The offsets of the objects in the flat-file implementation would then become the keys used to store the objects in LevelDB.

Data stores are permanently assigned to and tightly coupled with their hosting Storage Pool. Partly because larger systems will host a great number of data stores and partly because virtually all operations involving any single store will also involve several other stores within the same pool, stores are uniquely identified by their hosting pool's UUID plus the index of the store within the pool's array of data stores.

Storage Pool

Storage pools aggregate Data Stores into a logical units for object allocation, data dependency constraint, error recovery, and physical placement purposes.

Copyright © 2016 Cleversafe, Inc.

Object Allocation

Storage pools must contain a number of Data Stores matching at least the width of the widest IDA intended for use with the pool. Ideally it would also contain a few Data Stores beyond this minimum requirement in order to allow for allocations in the pool to succeed at full width even if one or two Data Stores are offline at the time of the allocation. The number of Data Stores should generally be kept small though as adding additional stores beyond the maximum supported width decreases the efficiency of the rebuild procedure for failed Data Stores.

Data Dependency Constraints

Storage pools can operate in almost total isolation from other storage pools. The Data Stores within the pool share object data with each other but never with stores outside the pool. This is advantageous for a number of reasons but is particularly important for the ability to accurately determine the health of the pool based on the health of its member stores. It is also helpful for determining whether or not it is safe to take a Data Store temporarily offline for maintenance.

Error Recovery

Aspen uses a self-hosting model for error recovery in which all data needed for recovering from physical media failures and missed object update transactions is stored in distributed tree structures constructed out of Aspen objects. The Error Recovery section goes into much greater detail but, essentially, there are two types of trees that are maintained for each pool. An **Allocation Tree** for maintaining *ObjectPointer*'s to all objects in the pool and **MissedTransaction** trees that contain an entry for each committed transaction where one or more peers are suspected of having failed to properly process it. These trees may be hosted entirely within the pool itself to maximize isolation or may be placed elsewhere for efficiency and/or durability reasons.

Physical Placement

For a geographically dispersed Storage Pool, the members of the pool may be spread out amongst the available physical locations. Alternatively, all members of a pool could be co-located at the same site in order to minimize access and update latency to the objects contained by those stores. Similarly, latency can be tuned by ensuring that all stores within the pool are backed by media with the appropriate access characteristics. Navigation of distributed trees, for example, benefits tremendously from ensuring that all of the upper branches of the tree are backed by solid state media.

For some workloads, data ages over time and becomes needed less frequently. As an option for effectively managing this, pools may be backed by fast media while the data is fresh and then migrated down to less-performant but cheaper hardware as it ages.

Transaction Model

A basic understanding of the Paxos protocol is necessary for understanding how the transaction model described here builds upon the core capabilities of the algorithm. The Paxos Refresher appendix provides a brief summary of the protocol and highlights the key aspects that are relevant to this section.

Aspen's transaction model provides immediately consistent, multi-object, atomic updates through the use of a single instance of the Paxos consensus algorithm for each transaction. In terms of concurrency control, this

Copyright © 2016 Cleversafe, Inc.

model could generally be described as Optimistic Concurrency Control (OCC) but there are some concepts blended in from a lock-based, 2-phase commit approach as well. Unlike many other transaction-based systems, there is no concept of "Rolling-back" aborted transactions in Aspen. Transactions are all-or-nothing operations with no intermediary results so aborted transactions simply fail to make any visible changes.

Each transaction is defined by a TransactionDescription that contains the following information (in Scala syntax):

```
class TransactionRequirement(
  commitQuorum:       Int,             // Number of positive votes needed to allow
                                       // 'Commit' to be chosen

  acceptingPeers:     Set[NetworkUID], // Data Stores hosting slices/replicas and act
                                       // as Paxos Acceptors for the transaction
  objectPointer:      ObjectPointer
)

class ObjectRevisionLock(
  requiredRevision:   ObjectRevision  // Current object revision must match this for the
                                      // transaction to succeed
) extends TransactionRequirement

class ObjectRefcountLock(
  requiredRefcount:   ObjectRefcount  // Current object refcount must match this for the
                                      // transaction to succeed
) extends TransactionRequirement

class ObjectDataUpdate(
  newRevision:        ObjectRevision, // New revision value if the transaction succeeds
  isAppend:           Boolean,        // True if appending data, False if overwriting
  repairRequired:     Boolean         // True if the MissedTransaction trees must be
                                      // updated for this transaction
) extends ObjectRevisionLock

class ObjectRefcountUpdate(
  newRefcount:        ObjectRefcount
) extends ObjectRefcountLock

case class TransactionDescription(
  // Uniquely identifies this transaction in the system
  uuid:                UUID,

  // HLC timestamp at which this transaction began
  creationTime:        Long,

  // Chosen from the set of all acceptingPeers
  designatedLeader:    NetworkUID,

  // Peers that only perform Proposer/Learner roles
  observingPeers:      Set[NetworkUID],

  // Actions to be preformed before the transaction may be forgotten
  finalizationActions: List[FinalizationAction],

  // List of requirements that must be met for the transaction to commit
  requirements:        List[TransactionRequirement]
)
```


Copyright © 2016 Cleversafe, Inc.

At the basic level, transactions define a list of requirements that must be met in order for the transaction driver to attempt to resolve a *Commit* decision and a list of cleanup activities that must occur before the participating nodes are allowed to forget the transaction's existence (delete all associated state).

The resolution process uses the standard Paxos algorithm but the *Prepare* and *Promise* messages are overloaded to carry some additional data beyond what Paxos itself requires. The `TransactionDescription` and slice/replica data is sent along with the *Prepare* message. Each participant's vote for whether or not the transaction should be allowed to commit is sent along with the corresponding *Promise* message. In order for the stores hosting slices/replicas of the target object to vote for transaction committal, they must first check their local state for all `TransactionRequirements` for which they host data. Only if all of the objects they host meet the transaction requirements may the store vote for committal. As the names imply, `TransactionRequirements` deriving from `ObjectRevisionLock` and `ObjectRefcountLock` also "lock" the target object attributes from modification while the transaction is outstanding if the store voted for committal of that transaction. Once the transaction has resolved to either a *Commit* or *Abort* decision, the store may again vote in favor of another transaction modifying those objects but until that point is reached it must vote to deny any transactions attempting to lock the already-locked object attributes.

The commit quorum may, and often will, exceed the minimum quorum requirement needed by Paxos. For an erasure-coded object, for example, the total number of slices may be twelve with eight required in order to reconstruct the object. At a minimum the commit requirement in this case would need to be set to eight to ensure that the updated object could be reconstructed. However, for reliability purposes, that number should probably be increased to ten or eleven to ensure that failures of one or more nodes will not bring the object below the reconstruction threshold. Additionally, each a commit quorum for `TransactionRequirement` must be achieved independently from each other requirement. If a commit quorum for any of the requirements cannot be achieved, the transaction driver must attempt to resolve an *Abort* decision.

Once the *Commit*/*Abort* decision is made, the remaining message exchange consists of standard Paxos *Accept* & *Accepted* messages.

Contention Mitigation

As with most Optimistic Concurrency Control (OCC) systems, transactions simultaneously attempting to update an object can conflict with one another and prevent any transaction from gaining a quorum of votes for committal. The naive solution is to abort any transaction that fails to achieve the write threshold and leave it to application-level code to retry the transaction at a later time. This works but it defers contention mitigation to the application layer and does nothing to ensure forward progress. A better solution is to abort all but one of the contending transactions. The remaining transaction may continually attempt to gain commit permission from stores that were previously locked to one of the contenders. As the contending transactions abort, the continuing transaction can make progress toward gaining commit approval.

A transaction driver can decide whether to continue or abort by examining the replies it received from the stores and applying the following rules:

1. If a write-threshold number of stores granted permission, commit.
2. If success is impossible due to revision mismatches on width - write-threshold stores, abort.
3. If no store granted commit permission, abort.
4. Examine the timestamps and object revision requirements of the other transactions for which stores granted commit permission. If this transaction has the lowest timestamp of all transactions with matching

Copyright © 2016 Cleversafe, Inc.

revision requirements, continue. Otherwise abort.

To prevent newly arriving transactions from interfering with the transaction that is continuing, each store should track the outstanding transactions on each object. When a transaction for which it has given permission aborts, it may immediately grant permission to the next outstanding transaction that has the lowest timestamp. Having the stores take an active role here also reduces the resolution latency that would otherwise be induced by having the continuing transaction repeatedly poll for approval. A polling mechanism is still required to ensure that a sufficient number of stores become aware of the continuing transaction but stores that are already aware of the transaction need not wait for the polling cycle to elapse before sending permission.

Optimistic Commits

Paxos normally requires four message delays in order to achieve consensus. Two delays in the network round trip to establish permission to propose a value and two more message delays in the second network round trip required to gain acceptance of that value. A long-standing observation of Paxos is that correctness is not threatened by locally generating and processing some of the required messages as long as the messaging restrictions are not violated. In certain circumstances this allows for a reduction in the typical four message delays required to achieve consensus.

Optimistic commits require three message delays to resolve a *Commit* decision by allowing the data stores to locally generate and process the *Accept:Commit* message. To do this, the *Promise* messages sent in response to the initial *Prepare* message are sent to all Data Stores in addition to the initial proposer. Each store then analyzes the promises from the other stores. If a Data Store sees that a write-threshold number of stores has voted for committal, it locally generates and processes the *Accept:Commit* message for the first round of the algorithm, thereby avoiding the message delay that would otherwise be required for the proposer to send the message over the network. In the failure-free case this allows consensus on *Commit* to be resolved in three message delays but comes at the cost of requiring a total of six message delays to resolve an *Abort* instead.

In the case where the commit threshold is not reached and the proposer wishes to abort the transaction, it must then send a new *Prepare* message for round 2 of the Paxos algorithm in order to gain permission to make a proposal for that round. This step may not be avoided as it is possible that one or more of the other peers on the network may have seen that the commit threshold was indeed reached and could have processed the *Accept:Commit* message for the first round. If that occurs, the proposer may be forced to resolve *Commit* in the second round.

The additional three message delays required to resolve *Abort* decisions could adversely impact performance when contention is high. Applications may consequently elect to disable optimistic commits at such times.

Returning Success to the Client in a Single Round Trip

Although full resolution requires three message delays (1.5 round-trips) in the optimistic commit model, success to the client may be returned in a single round-trip if sufficient 'Promise' messages are received. In the simple N+1 quorum scenario, the sufficient number of responses follows the Fast-Paxos requirement of a 2/3rds majority. For erasure-coded information dispersal algorithms or for replication factors above a simple majority, the required percentage of responses increases to ensure that no sufficiently large group of non-responding stores could sway the consensual decision.

Copyright © 2016 Cleversafe, Inc.

Finalization Actions

Aspen expands upon the Paxos algorithm's requirement of saving a small amount of state to a stable storage medium during the resolution process. In addition to saving the required state to ensure correct consensus, Aspen also requires that the *TransactionDescription* and pending slice/replica updates survive system crashes and power outages. This protects the data and transaction models from corruption and, equally as important, it protects against the loss of any *FinalizationActions* contained within the *TransactionDescription*. *FinalizationActions* are idempotent actions that must be completed before peers are allowed to delete the state associated with a transaction and they are executed by any peer that drives the consensus algorithm to resolution. They are particularly important tools for ensuring that all necessary error-correcting information is captured by the system but they are also general-purpose utilities with many potential uses.

At a minimum, each transaction carries an implicit *FinalizationAction* to create an entry in the *MissedTransaction* tree if one or more peers are suspected of having failed to properly process the transaction. Entries made in *MissedTransaction* trees allow for recovery from temporary offline periods and transient network failures. Another essential *FinalizationAction* applies to all allocation transactions and ensures that a pointer to the newly allocated object is recorded in its StoragePool's *AllocationTree*. When a Data Store fails and must be reconstructed, every object with slices and/or replicas hosted by that store may be discovered by walking the allocation tree.

As mentioned in the section on distributed data structures, *FinalizationActions* are also useful for maintenance activities that, while required, need not be atomic and part of the transaction. By moving non-essential object updates out of the transaction and into finalization actions, the potential for update contention is reduced so the transactions are correspondingly more likely to succeed.

The requirement of writing to stable media twice for each transaction is an obvious performance bottleneck. There are some mitigation options available though. NVRAM is the obvious first-choice for a backing media of transient transaction state but even with its rapidly falling price point it is unlikely that pure-NVRAM solutions will be cost effective. A tiered model where transaction state is initially stored in a relatively small area backed by NVRAM and then migrated down to a transaction log backed by SSD will likely be sufficient for most workloads.

Transaction Participant Restrictions

For error recovery purposes, transaction participants are restricted to the Data Stores hosting the objects being updated. An arbitrary number of additional entities may observe the messages involved in the resolution process, thereby acting in the *Learner* capacity of the Paxos protocol, but they are not permitted to act as Paxos *Acceptors* or *Proposers* with the single exception of sending the initial *Prepare* message. When sent by a non-transaction participant, the *Prepare* message will include an "initial proposer" field that designates one of the legal transaction participants as the initial proposer/transaction driver. When that peer receives the *Prepare* message, it will immediately take over the *Proposer* role and drive the transaction to closure. As mentioned in the error recovery section, the purpose of this is to allow for more efficient, batched updates of the *MissedTransaction* trees.

Peer Failure

If the peer driving the transaction to consensus fails before consensus is reached, one of the other peers must step in to complete the job. Similar to most other Paxos-based systems, Aspen uses a mechanism based on

Copyright © 2016 Cleversafe, Inc.

failure detectors to continually monitor the liveness of data stores (the monitor will, of course, actually track the liveness of the network endpoints hosting the stores but that's mapped down to data stores to avoid the need for introducing another architectural concept). While a store is live, it is left alone to process all transactions it is the designated leader for. When failure of that store is detected, each participant chooses a random timeout from within a small window at which point it will begin driving the transaction to closure if no other peer has started to do so.

Peer Interference

Peer interference is largely mitigated through the failure model which attempts to ensure that only one peer is ever attempting to drive the transaction to closure. However, it is possible for multiple peers responding to the failure of the initial leader to begin driving the transaction at approximately the same time. When this occurs, peers use a simple exponential back-off mechanism to ensure that one of them eventually succeeds.

Distributed Data Structures

Distributed data structures are formed by embedding serialized *ObjectPointer*'s in Aspen objects. Similar to most programming languages with reference-counted objects, Aspen supports both strong and weak references to objects where strong references increment/decrement the reference count of their pointed-to objects when they are created/desroyed and weak reference do not. There is no physical distinction between the two so it is up to the application logic to determine which type of reference each pointer is. Objects also do not have a special interface for dealing with embedded object pointers so it is up to either the Aspen system itself or the immediately superior application layer to ensure that the embedded pointers and reference counts are handled appropriately. In particular, the agent attempting to initiate a transaction that will set the reference count of an object to zero, thereby deleting it, must ensure that the all of references within that object have been properly dealt with or that provisions have been made to ensure that they eventually will be.

Efficient handling of embedded reference counts during node deletions can be a bit challenging. Consider, for example, a tree structure with several million nodes where each node holds strong references to the nodes immediately below it and each node has a reference count of one. What happens when the reference to the root of the tree is removed? The straight-forward solution of recursing through the tree and doing a bottom-up removal of nodes is impractical since the machine doing the removals could crash at any point, effectively canceling the operation. Similarly, even if machine reliability were not a problem, using only a single worker to delete a very large data structure would be quite slow. Ideally, multiple machines would cooperatively work through the deletion process.

There is no general-purpose solution that is ideal for all use cases but many approaches can be taken. A few of the available options are:

- Create a task to ensure that the deletion process has a persistent, crash-proof leader to coordinate the process. It may spawn sub-tasks for deletion operations with appropriate degrees of parallelization
- Move the pointer to the root node into a deletion tree which persistent worker tasks continually monitor for new entries. When a tree to be deleted is seen, they randomly select a bottom-most node and attempt to remove it. Once all nodes are removed, the deletion is complete.
- Cheat. If the tree is isolated to a set of StoragePools that host no other data structure, delete the pools instead.

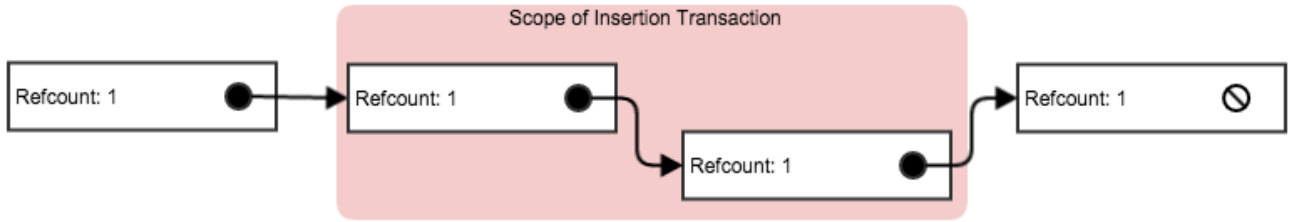
Copyright © 2016 Cleversafe, Inc.

Singly Linked List

Singly linked lists of Aspen objects operate in the traditional manner and have few consistency concerns due to their simple structure and the atomic nature of the update transactions. The two primary issues to consider with linked lists is management of the reference counts on list members and whether or not transactions attempting to remove objects from the list should fail if the version of the object being removed has changed from an expected value.

Although there may be some use cases for linked lists based on weak references that do not increment the reference count of its constituent objects, most lists will employ strong references. Assuming strong references are being used, the reference count for an object only needs to be modified when it is inserted into and removed from the list. The embedded pointer to any particular node may migrate around as list insertions and deletions occur but those updates are done atomically so the embedded pointer effectively exists in only one object at a time. Insertion of an object into the list would perform three operations in one atomic action:

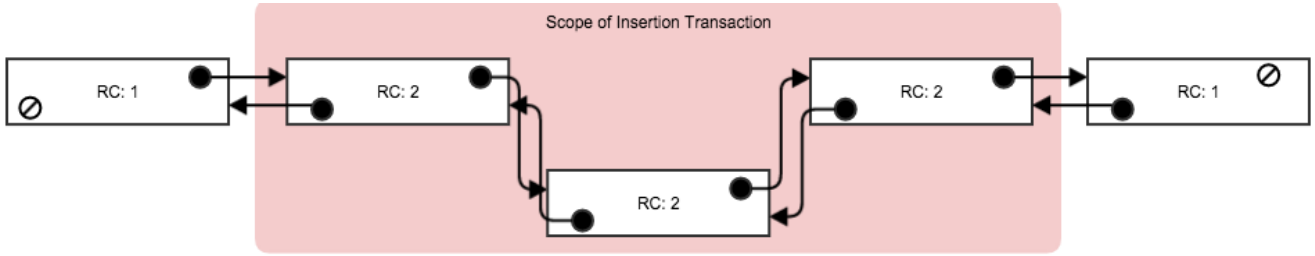
- Copy the *next* pointer of the left node into the new node
- Change the *next* pointer of the left node to point at the new node
- Increment the reference count on the new node by 1



The second issue concerning removal failures when the revision of the node being removed differs from the expected value depends upon the use case. As an example of why failure in this case can be desirable, consider a Last-In-First-Out queue where the processor of the queue has consumed all of the entries in the last object in the list. The processor would then initiate a transaction to prune the final object from the list. Even if the processor first re-reads the object to check and see if additional elements had been added since the last time it read the object, there will still be a window of time between the completion of that read and the completion of the removal transaction. Items inserted into the queue during that time would be lost. This problem is avoided though if a revision lock on the being-removed node is made a transaction requirement.

Doubly Linked List

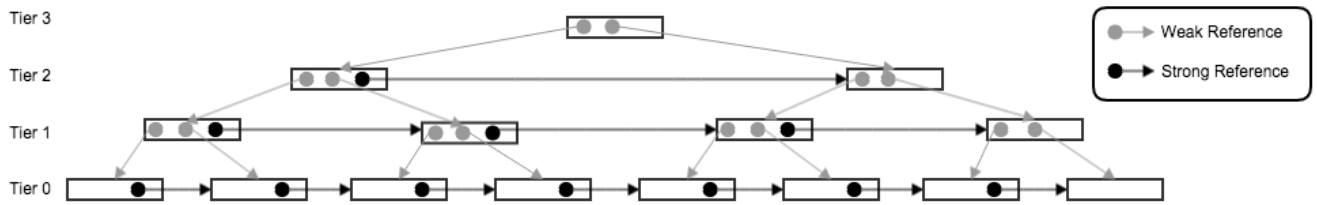
Doubly linked lists function in a similar manner to singly liked lists except that three nodes must be updated simultaneously:



Copyright © 2016 Cleversafe, Inc.

Tree

There are many potential ways to implement trees on top of Aspen objects. A simple method with surprisingly good contention and consistency characteristics is using tiers of singly-linked lists where the bottom tier stores data and the upper tiers store only weak-references to the nodes in the tier below it. As the tree grows in size, new tiers are added above what was previously the top tier. The notation used here is that tier-0 is the bottom tier and tiers-1+ refer to the tiers above it.



For key-value trees, each node in the tree has an immutable `minimum` attribute which specifies the minimum allowable key that may be stored in or below this node in the tree. This value is embedded within all object pointers that point to the node and allows for trivially simple navigation. When searching for an object from the top of the tree, simply follow the pointer with the largest minimum value that is less than the target value.

Because singly-linked lists are consistent in terms of navigation, navigating the data stored in the tree from left to right is guaranteed to be consistent, as is each tier of the tree in isolation. Navigation between the tiers, however, is not guaranteed to be consistent if multi-object transactions are not used to simultaneously update the nodes left of and above the node being inserted. While using multi-object transactions for this purpose is certainly possible, the stronger consistency guarantees will usually not be worth the additional update contention. If the references to inserted nodes are placed in the upper tiers through *FinalizationActions*, which happen out-of-band with and shortly after completion of the transaction that does the insertion, there is a small window of time in which the newly inserted node will be present in the lower list but not in the upper. When navigating the tree, however, this is generally not a significant problem as the error is easily detected and may be corrected by simply scanning to the right until the target node is found. This mode of navigation is similar to the approach publicized in 'Efficient Locking for Concurrent Operations on B-Trees' by Lehman and Yao. An additional benefit of being able to detect and easily work around such inter-tier inconsistencies is that stale, cached copies of the upper tiers of the tree will not cause lookup or insertion failures. At worst, out-of-date copies will induce a small performance penalty.

A slight enhancement upon the basic tree model that allows for efficient storage of both large and small values in the tree is to support *ObjectPointer* values in addition to raw data. If the size of the value exceeds a certain threshold, a new object could be allocated to store the value and a strong pointer to the object could then be inserted into the tree. A similar mechanism where weak references are used instead could be used for indexing collections of objects for searching purposes. As with all cases in which embedded references are used, care must be used to ensure that the reference counts are handled appropriately.

With repsect to the data stored in the tree, there are two basic mechanisms that can be used. The nodes in tier-0 of the tree may be entirely overwritten with each update or they may be incrementally updated with append operations. The advantage of the former approach is that it entails minimal processing and storage overheads but comes at the cost of additional network overhead from re-sending already stored data along with each update. The costs and benefits to the latter approach are effectively the inverse of this. Whereas the overwrite-based approach stores the exact state of the tree at any given point in time, the append-based model stores a log of operations. So where the serialized content of the former approach may be **(A,B,D)**, the equivalent

Copyright © 2016 Cleversafe, Inc.

content in the latter may be **(Insert:A, Insert:B, Insert:C, Insert:D, Remove:C)**.

Graph

The techniques used for lists and trees can be extended to the construction of arbitrary graphs of objects. The primary concern, as with any reference-counted graph structure, is the proper maintenance of the reference counts as embedded pointers to nodes and edges are added and removed. The specific mechanisms used will be heavily dependent upon the specific use case though. Unlike key-value trees which are fairly general-purpose in nature, most graph-based solutions will likely be specialized for specific use cases.

Optimizations

Leveraging Storage Pools

Storage pools are one of the primary knobs for tuning performance, reliability, and availability. The following sections describe some of the primary use cases.

Object Isolation

Storage Pools may be used to isolate related groups of objects into distinct sets for various purposes. This is often useful for data management purposes whereby related groups of objects may be migrated between machines or media to satisfy availability or performance needs. Isolation can also be useful for supporting special use cases such ultra-fast deletes of data structures by deleting their supporting storage pools rather than iterating through each element of the distributed structure.

Isolation can also be an enabler for optimizing distributed data structures. Trees, in particular, can greatly benefit from using distinct pools for objects in the upper tiers of the tree. Other sections will go into further detail but a number of optimizations become possible by isolating the upper and bottom tiers of the tree from one another.

Geographic Placement

Storage pools can be used to ensure that slices and/or replicas are distributed between multiple sites to achieve the desired reliability, availability, and latency characteristics. Beyond simply spreading the data around for disaster recovery purposes, intelligent use of replication of objects at specific sites can also be a performance enhancer for certain workloads. In particular, reads of replicated objects can avoid expensive over-the-wan messages when reads of potentially out-of-date objects is acceptable.

Workload Separation

Pools may be used to ensure that their constituent stores are isolated to distinct sets of physical machines. A primary use case for this is workload separation. This could be used, for example, to restrict all of Aspen's internal support structures and data processing activities to a small set of high-performance systems on an isolated network to ensure that those activities do not adversely affect end-user applications. It could also be leveraged to isolate end-user workloads from one another perhaps for Quality of Service reasons or for enforcing physical maintenance boundaries.

Copyright © 2016 Cleversafe, Inc.

Media Placement

Pools may be leveraged to ensure that groups of objects are backed by the appropriate physical media that matches the latency vs storage expense needed for the data they contain. Frequently accessed objects would benefit from flash media whereas a pure archival application may prefer to write objects directly to tape.

Migration

As the stores within pools have no fixed location, they may be migrated to suit the changing needs of the application environment. Migrating data between geographic regions may make sense for a use case where there are more sites than replicas and it is desirable to shift an application from one location to another. In such a case the application would likely benefit from migrating its supporting data to the same location to keep latencies down. Likewise, as data ages over time, migrating it from expensive, high-performance media down to slower but more cost-effective media may be advantageous.

Erasure Coding vs Replication

The primary advantages of erasure coding over replication is storage efficiency and the network bandwidth needed for writes. Whereas five-way replication requires an additional 400% of both, erasure coding for the same level of reliability requires only an additional 60%. The primary drawbacks are expensive rebuild operations and the inability to do local processing of object content due to that content being sliced up and spread across multiple hosts. The pros and cons of replication vice erasure coding are essentially the inverse of this. The storage cost and write requirements are significantly higher but reconstruction operations are cheap and processing of object content may be done by the stores that host the copies.

Distributed data structures may mix and match replication with erasure coding to balance the tradeoffs. For example, tree structures will almost always benefit from using replication for tiers-1+ instead of erasure coding. The ability to locally process object content at each store can greatly cut down on the latency and network bandwidth required for navigating the tree structure.

Consider the steps required for a fully erasure-coded tree that is five levels deep and comprised of 14Kb objects in tier-0 and 80Kb objects in tiers-1+. Reads require five round-trips and 334Kb of data to be sent over the network. For a similarly configured tree that uses replicated objects in tier-1+, that can be reduced to an effective three round-trips and 14Kb of data sent over the network by taking advantage of the local processing capabilities. Rather than navigating the tree directly, a searcher may instead request that the hosting stores do so in its stead. The request is sent to a store that hosts object at the top of the tree. That store reads its local copy of the object, determines the next hop in the link and forwards the navigation request to that node, and so on until the node in tier-0 is found. The store hosting that object may then respond directly to the originator of the request. The message pattern in this case looks like: Originator -> StoreA -> StoreB -> StoreC -> StoreD -> StoreE -> Originator

Using replication in this case reduces the network bandwidth needed for tree navigation by 96% and two round-trips over the network, which can be particularly beneficial for geo-dispersed systems. An additional benefit of upper-tier replication for geo-dispersed tree structures is that the replication mechanism can be configured to ensure that replicas of all tier-1+ nodes are available at each site, thereby avoiding the need to pay a 90ms+ navigation penalty for each tier in the tree. The total storage requirement for the upper tiers is approximately 1% of the total tree size so LAN-speed navigation of storage-efficient, erausure-coded, geo-dispersed tree structures can be achieved at the cost of an additional storage overhead of 1% per site.

Copyright © 2016 Cleversafe, Inc.

Physical Media

Storage pools may be used to isolate clusters of objects and ensure that the data stores hosting them are backed by media with the appropriate latency characteristics. Frequently accessed and/or frequently updated objects would benefit from being backed by low-latency, non-rotating media wheras objects that do not have stringent latency requirements would be better placed on less performant but correspondingly less expensive media.

Use cases that are subject to data aging may take advantage of the ability to migrate data stores between backing media and network endpoints. For example, a warehouse for sensor data may require fast access to data collected in the last few weeks, medium access to data over the last 90 days, and very infrequent and slow access to data older than that. If the data is stored in tree structures, new storage pools may be created each morning to host the day's sensor data and the corresponding pool at the migration threshold may be migrated down to slower storage: ssd -> hdd -> tape.

Some data structures can also leverage physical media to their advantage. Trees, in particular, benefit tremendously from tier-1+ objects backed by non-rotating media. A five tier tree backed entirely by disk drives with 7ms latency will entail a media-induced overhead of 35ms per lookup whereas using ssds with 0.3ms latency for the same tree drops that overhead to around 1.5ms. Combining the two approaches with ssds for tiers-1+ and disk drives for tier-0 comes out to about 8.2ms, a value only marginally above that of the hard drives alone.

Concurrency & Consistency

Multi-Version Concurrency Control (MVCC)

MVCC can be implemented in a number of ways on top of Aspen's object model. In each case though, explicit support must be added in to the distributed data structure requiring it. One possibility for efficiently maintaining short-duration MVCC histories for key-value trees would be to maintain an inverse history log for each node in tier-0 of the tree. Every entry in the tree, both additions and deletions, is given an HLC timestamp and multi-object transactions are used to simultaneously insert the new entry into the tree along with the inverse of that operation into the log. This would allow the previous state of the node to be recovered by reading its current state and applying all entries in the inverse history log to effectively rewind all changes made to the node.

The logs could be implemented as slight variant of doubly-linked lists to allow for simple navigation in either direction and a pointer to the oldest object in the log could be placed into a history-pruning tree. The key for this tree would be comprised of the oldest HLC timestamp held in the log with the object's UUID appended to it. The sorted-order nature of the tree would ensure that all of the oldest log entries would be easily obtainable and pruning the logs of expired content would require nothing more than deallocating the oldest node in the linked list followed by updating the history-pruning tree.

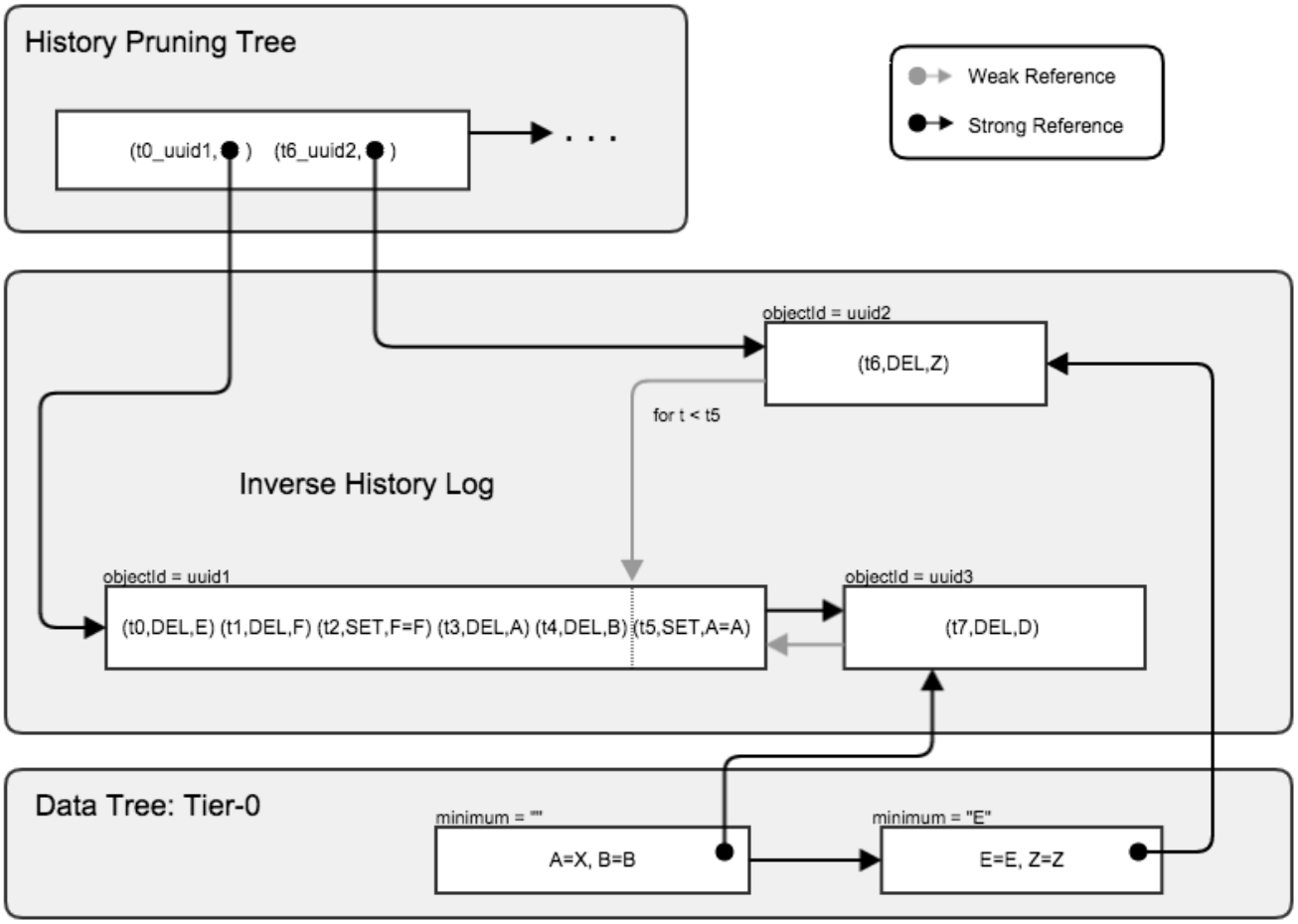
The inverse history log maintained for any individual node appears linear when navigating backward in time. Iterating in the forward direction, however, will show only the history for that specific node and will make no mention of node splits. When a split occurs, a new log object is allocated that points to the log it derives from but `next` pointer in the previous log is not updated to point at the divergent log.

For example, consider the following sequence of events applied to an empty key-value tree:

Copyright © 2016 Cleversafe, Inc.

1. Insert: E=E
2. Insert: F=F
3. Delete: F
4. Insert: A=A
5. Insert: B=B
6. Insert: A=X *(A branch split occurs here with "E" being migrated to the new branch)*
7. Insert: Z=Z
8. Insert: D=D

This would result in the following inverse history structure:



For each node in the data tree, the previous state for that node can be reconstructed by applying entries in the inverse history log back to the desired point in time, even if the node didn't actually exist at that point in time. Only those operations that affect key-value pairs that fall within the key-range of the node are applied.

A potential optimization with this approach would be to isolate the storage pools hosting the log entries and history-pruning trees to a separate network backed by SSD media. The data it stores would be transient, frequently updated, and limited in size (approximately history window times the maximum ingest rate).

Copyright © 2016 Cleversafe, Inc.

Read-Atomic Isolation

Read-Atomic (RA) Isolation is an approach to reading distributed objects that ensures that either all or none of the updates associated with transactions operating on those objects are visible at once. This property can be useful for implementing applications, such as MongoDB backends, that have strong read-consistency requirements. In the case of MongoDB backends, it is required that all secondary indices referencing an object must be atomically modified with the update to the object itself as inconsistencies between the indices and the objects is not permitted. RA reads provide a lightweight mechanism for ensuring that the returned objects are consistent with one another even in the presence of concurrently executing write transactions that span multiple sites.

The paper linked above goes into greater detail but essentially RA reads are accomplished by reading the current state of all outstanding transactions operating on a set of objects in addition to reading the objects themselves. The acquired object revisions and transaction state are then analyzed to determine if the read set of objects are consistent with one another and, if not, additional reads are preformed until a consistent set is obtained. This approach is a good match for Aspen's transaction model as all of the relevant information needed for implementing RA reads is contained in the TransactionDescription.

Internal Support Structures

Aspen is a self-hosting system in that almost all system management and error recovery information is stored within the system itself. The following sections describe the internal data structures and how they're used.

Master Tree

The internal support structures are built on top of distributed trees, the root of which is the MasterTree. This tree contains pointers to the left-most node of every tier of every system-level tree, including those of the master tree itself. The resulting structure looks like the following:

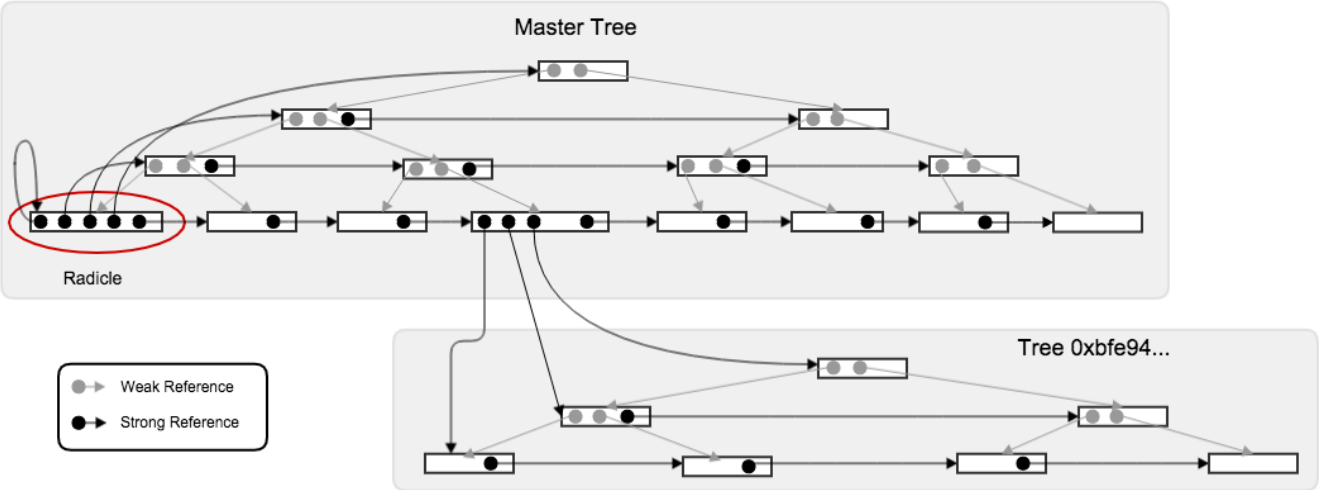


Figure 1. Tree Structure

All trees are identified by a random UUID with the single exception of the master tree which instead uses a zeroed UUID. The keys in the master tree are formed by appending a single byte to the UUID of the tree which specifies the numeric tier of the tree it points to and the value is the object pointer to the left-most node of the

Copyright © 2016 Cleversafe, Inc.

tier. The metadata associated with the tree is also inserted into the master tree with key UUID + 255. The natural batching of the keys means that tree lookups will need to navigate at most one additional node to the right in order to find the top-most node of the target tree and its associated metadata. Using a zeroed UUID for the master tree ensures that pointers to the master tree's upper tiers are sorted into the left-most node of its tier-0 linked list. This node, known as the *Radicle*, is the ultimate root of the storage system; every object stored in an Aspen system can be found by following pointers from this point of origin. Consequently, bootstrapping an Aspen system requires only an offline copy of the object pointer to the Radicle node and the configuration of the initial storage pool. All other system support and management information is stored within the system itself.

Task Tree

This tree maps task UUIDs to pointers to objects that contain the description and state machines of long-running system tasks such as reconstructing failed data stores and the recursive deletion of large tree structures. The task model is further elaborated upon in the distributed tasks section.

Storage Pool Configuration Tree

This tree maps storage pool UUIDs to pointers to the objects that contain their current configuration. The configuration contains at least the following information:

- Network endpoints for each member store
- Maximum supported IDA width
- IDA to use for the AllocationTree and all MissedTransaction trees
- The UUID of the AllocationTree for the pool
- The UUID of the MissedTransaction tree for each member store
- Maximum slice/object size

Storage Pool Support Trees

Allocation Tree

The purpose of the AllocationTree is to facilitate the reconstruction of lost data stores by maintaining a weak pointer to every object in the pool. Entries in the AllocationTree are maintained by FinalizationActions and therefore the content at any given point in time may be slightly out of date. A fully up-to-date view of the allocation tree can be obtained by querying all outstanding transactions and overlaying that information on top of the current state of the tree.

Contention mitigation and buffering techniques are used to support efficient updates to the AllocationTree when members are concurrently allocating objects at a high rate. Both issues are addressed by ensuring that the keys used for the allocation tree use a peer-specific prefix and sort in sequential order for objects allocated by that peer. The key and object UUID are derived via the following process:

1. Allocate a random UUID
2. Replace the top-most bits with the pool member index

Copyright © 2016 Cleversafe, Inc.

3. Replace the bits below that with the HLC timestamp of the allocation (each allocation is considered as an event from the HLC perspective and will cause the *c* value to increment, thereby ensuring there are no duplicates)
4. The resulting value is used as the key into the AllocationTree
5. The UUID of the object is set to the xor of the StoragePoolUUID and this key

When attempting to look up the entry for an object in the AllocationTree, the key may be derived by xoring the object UUID with the storage pool UUID. With this approach, there will be a short, initial window of time in which stores will experience contention in updating the AllocationTree but this will vanish once each store has induced a branch split. All subsequent allocation updates will be contention free.

Missed Transaction Trees

Missed transaction trees are maintained on a per-member basis and are treated in a log-like manner. Whenever a pool member drives a transaction to completion and it suspects that one or more peers failed to properly process the transaction, which will generally be those peers for which it failed to receive an *Accepted* response for the consensual value, an entry for that transaction will be placed in the Missed Transaction Tree owned by the driving peer. Allocations always require 100% success so the only entries that will be placed in this tree are missed attempts to delete objects or update their content. Missed updates to object reference counts do not need to be logged since they are replicated along with each object slice/replica and include a monotonically increasing update number along with the reference count. Simply querying a majority of slices/replicas is sufficient as the reference count with the highest update number is guaranteed to be the correct value.

Keys for this tree are formed from the timestamp of the entry plus a small serial number to prevent duplicates. Each peer tracks the MissedTransaction logs of its other peers and tracks the oldest timestamp for which it knows it has processed all log entries for each peer. Periodically peers exchange these values and prune old entries in their logs to keep the missed transaction trees from growing indefinitely. The values are three-element tuples consisting of:

1. Enumerated value indicating whether the entry is a missed update or missed delete
2. Bitmask with 1's for each peer suspected of having missed the transaction
3. List of pointers to the objects in question

The purpose of using separate MissedTransaction trees for each peer is to enable contention-free, batched updates. This is effective for single-object transactions where the transaction driver is guaranteed to be a peer on the pool owning the object. For multi-object transactions, however, the driving peer will often not be a member of the pool. In that case it may ask one of the peers hosting the object in question to add an entry to their missed transaction log. Once the driving peer has received a positive acknowledgment of all such entries, it may consider the finalization action complete.

Missed transaction trees are normal trees in most respects but they do have a unique property in that the transactions used to update them do not themselves trigger updates to the Missed Transaction tree as doing so would cause cycles while stores are offline. Consequently, the replication factor for nodes comprising the MissedTransaction trees should provide a higher level of reliability than the regular data stored within the pool. Specifically, the write-threshold requirement should equal or exceed the highest-reliability IDA allowed in the pool.

Copyright © 2016 Cleversafe, Inc.

Distributed Tasks

FinalizationActions provide a limited form of distributed task management in that they ensure that the follow-on actions that need to be preformed before a successful transaction may be forgotten are replicated across a write-threshold number of nodes. They also ensure that those nodes will detect and recover from processing failures. This level of distributed tasking is appropriate for relatively simple and short-duration activities but a more robust mechanism is needed for complex, long-running tasks such as reconstructing failed stores and deleting large data structures.

An additional requirement on the tasking model for some necessary maintenance activities is that it must support "exactly once" operations. "At most once" and "At least once" guarantees (which are provided in this system by transactions and finalization actions, respectively) are relatively easy to achieve in distributed systems but "exactly once" guarantees are much harder to come by. A third party utility could be used to provide this capability but a major advantage of implementing the tasking model directly within Aspen is that by storing the state for the distributed task within an Aspen object, updates to the task state may be tied directly into the transactions used to carry out the task. For example, if a task needed to decrement the reference count on an object exactly once, the task could use a multi-object transaction to simultaneously decrement the reference and update the task state to reflect that the reference has been decremented. Even if multiple processing agents were attempting to do so simultaneously, all transactions after the first one would fail to commit since the revision of the object holding the task state would not match the transaction requirements.

Building a robust task management system that knows how to appropriately distribute load and detect and recover from the innumerable problems present in distributed systems will be a significant challenge in its own right. To keep focus on the core architecture, the rest of this document will assume that a robust solution exists and that updates to task state may be tied into the transactions used to carry out the task's steps. Beyond that point, the task management system is left unspecified.

Error Recovery

Recovering from Offline Periods and Transient Network Failures

As mentioned in the description of the MissedTransaction tree, each data store continually monitors the logs of the other stores in its pool to discover transactions that it missed. Each peer locally tracks a timestamp behind which it knows that there are no entries in any peer's missed transaction tree that it hasn't yet taken corrective action for. Entries later than this timestamp are searched for in each peer tree and corrective action is taken as entries are found. As new times are discovered behind which all entries have been successfully recovered from, the local timestamp is advanced. Periodically the peers in a pool will exchange their local timestamps and will use the values to determine how far back in time they must maintain entries in their trees. Entries prior to this time may be pruned to keep the trees from growing indefinitely.

When an entry is discovered in a peer's tree that has the bit set indicating that the scanning peer missed the transaction, it uses the embedded pointer to look up it's local slice/replica and then queries the current revision of the object from the other peers. If the revisions match, no corrective actions are required (the entry was probably a duplicate). If they do not match, the correct local state is reconstructed by reading from the other hosts and the local state is updated to the correct value.

Copyright © 2016 Cleversafe, Inc.

Recovering Lost Data Stores

When the physical media backing a store is lost, the basic recovery process involves recreating the failed store from the content of the pool's allocation tree. Tier-0 of the tree is walked left-to-right and each object pointer is inspected to see if the failed store hosted any slices/replicas of that object. If so, the required local state for the failed store is reconstructed from the state held by the other slice/replica hosts and written to the recovering store. As this will generally be a long-running operation that must be completed correctly for long-term system reliability, it should be implemented in terms of the distributed task model.

Although the basic process is almost trivially simple, there are a great many tradeoffs and optimizations that practical implementations need to consider. A few of which are:

- Should the store be reconstructed on the same physical machine or somewhere else on the network?
- Optimizations for replicated as opposed to erasure-coded objects
- Optimizations based on local vs geo-dispersed objects
- Optimizations for fixed-size flat-file backing store implementations where reconstructing objects in the proper order eliminates seeks and maximizes write throughput.
- Potentially using physical media shipped between geo-dispersed sites rather than relying on WAN bandwidth

Transaction Crash Recovery

There are two goals to transaction recovery:

1. Ensure that the transaction resolves a *Commit/Abort* decision
2. Determine whether or not to apply the pending object updates to local state (if any)

The recovery process for a transaction begins by asking a threshold of stores from each transaction requirement for their view of the current state of the transaction as well as the current revision, reference count, and last committed transaction UUID for the object targeted by the transaction requirement. The following rules are then used:

1. If the transaction is still in-progress, resume processing.
2. If the transaction committed/aborted and is still awaiting completion of the finalization actions, locally apply/discard the object modifications (if any) and resume processing.
3. If the peers return a cached commit/abort result, act accordingly and discard the transaction state
4. If the current object revision does not match the required value for one or more of the transaction requirements, compare the transaction UUID with the UUID of the transaction that last modified the object. If they match, commit the local modifications. Otherwise, discard the local slice/replica and repair the object.
5. Bump the object revision and then discard the transaction state.

Various patterns of network partitions and dropped messages conspire to make it very difficult to distinguish between an aborted transaction and a transaction that is in-progress but has not yet reached a threshold number of peers. This recovery mechanism bypasses the problem with the last step of bumping the revision of

Copyright © 2016 Cleversafe, Inc.

the object which makes it impossible for the potentially outstanding transaction to commit. Once the object revision changes and the last committed transaction UUID does not match the transaction in question, the recovering transaction may be silently discarded.

Object Allocation Crash Recovery

Object allocations are tied to a transaction but the stores that host slices/replicas for the newly allocated object are not, themselves, part of that transaction. Because each store must provide a local reference to the slices/replicas hosted by that store, the *ObjectPointer* for the new object and, therefore, the *TransactionDescription* for the transaction used to insert the new pointer into an existing object cannot be formed until after the initial allocation step. The normal flow to resolve allocations is for the initiating store to immediately inform the allocating stores if they failed to achieve the required 100% allocation threshold, thereby abandoning the transaction before it even began, or to include a *FinalizationAction* in the *TransactionDescription* that will attempt to notify all allocating stores of the success/failure of the transaction. If an allocating store goes offline after the allocation step and comes back online after the transaction has been forgotten, it must determine what to do with the local slice/replica.

Although the full *TransactionDescription* cannot be formed until after the initial step, the transaction UUID can be determined in advance as can the object that will be updated in the allocation transaction. This information is sent to the allocating stores along with the object data to assist in the recovery process. The process begins by querying a threshold number of the stores hosting the object-to-be-updated for the current state of the transaction and the current state of the object. The recovery steps are then:

1. If the transaction is still outstanding, await notification from the *FinalizationAction*
2. If a cached commit/abort decision is available, act accordingly.
3. If the last committed transaction UUID on the object matches the one for the allocation transaction, commit.
4. If the revision of the object does not match the required value, check the StoragePool's AllocationTree. If a pointer to the object exists, commit. If it does not exist, abort. (The *FinalizationAction* that updates the AllocationTree ensures that if the transaction committed a recovering store will either see the outstanding transaction state or an entry in the Tree unless the object is deleted before the store recovers, in which case throwing away the allocated state is also legitimate)
5. Otherwise the object revision matches the expected value. Similar to transaction recovery, attempt to bump the object revision. If the revision changes for any reason and the last committed transaction UUID does not match the value required by the allocation transaction the local state may be discarded as it is impossible for the potentially-outstanding allocation transaction to commit.

Paxos Refresher

A full and relatively easy-to-read description of Paxos may be found 'here'.

Paxos is an algorithm for achieving consensus on a single value over unreliable network channels and it does so through a two-step process:

1. Obtain permission to propose a value from a majority of peers via *Prepare* and *Promise* messages
2. Obtain acceptance of the proposed value from a majority of peers via *Accept* and *Accepted* messages

Copyright © 2016 Cleversafe, Inc.

The key to the algorithm's correctness is that the choice of the value to propose in the second step is constrained by the replies received from the first step. If an only if the messages granting permission to propose a value indicate that no other value has already been accepted by any peer may the proposer choose any value to propose that they wish. If, instead, some value has already been accepted by even one peer, the proposer must propose the previously proposed value. There is a bit more to it than that, of course, but for the purposes of this document it's sufficient to understand it as: "If you're first, propose whatever you want. If you're not, go with what the first guy said".

The two-step process can fail at either step and the recovery mechanism is to simply try again from the beginning. Rounds of the algorithm are distinguished with by a *ProposalID* which is essentially a counter that increments each time a peer tries to start over. Typically this is implemented as a tuple of `(counter, peerUID)` where the `peerUID` is included to ensure that the *ProposalID* is unique (two nodes are not allowed to use the same proposal id). To prevent interference between rounds, Paxos uses a really simple algorithm. If the *ProposalID* attached to a message is equal to or greater than any previously seen, process it. Otherwise ignore it (though practical implementations typically send a Nack rather than just ignoring messages).

The first step consists of sending a Paxos *Prepare* message to all peers which then reply with a *Promise* message (or *Nack* if the *ProposalID* is too low). Assuming the driving peer receives a majority of *Promise* messages, it will then choose a value (either the one it wants or the one it was forced to select as a result of being second) and send an *Accept* message with the value included in it. The peers will then reply with an *Accept* (or, again, a *Nack* if the *ProposalID* is too low). Assuming a majority of *Accept* messages are received, consensus is reached on the proposed value. If anything goes wrong, peers just increment the counter in their *ProposalID* and try again. Ad infinitum.

One important point to note is that durability in the face of crashes/power outages is achieved in Paxos by requiring that a small amount of state be written to persistent media prior to sending each *Prepare* and *Accepted* message.

There are two additional aspects to Paxos that are left unspecified by the algorithm itself but which must be taken into account by all practical implementations:

1. Detecting and responding to peer failure (i.e. the peer driving resolution forward crashes mid-process)
2. Mitigating peer interference (Multiple peers simultaneously attempting to drive to resolution could stall progress indefinitely)

Copyright © 2016 Cleversafe, Inc.